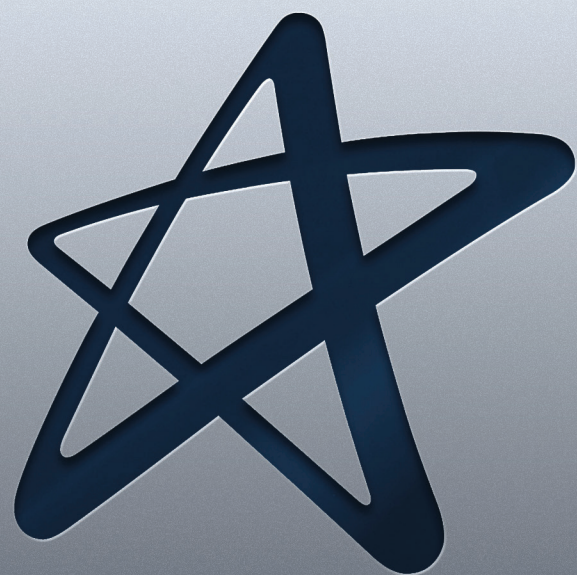


Estruturas de Dados Lineares



Cruzeiro do Sul Virtual
Educação a distância

Material Teórico



Estruturas de Dados Estáticas

Responsável pelo Conteúdo:

Prof. Dr. Luciano Rossi

Revisão Textual:

Prof.^a Esp. Kelciane da Rocha Campos

UNIDADE

Estruturas de Dados Estáticas



- Introdução;
- A Memória;
- Vetores;
- Fila Estática;
- Fila Estática Circular;
- Pilha Estática.



OBJETIVOS DE APRENDIZADO

- Apresentar ao aluno diferentes formas de organização de dados em memória, considerando os pontos fortes e limitações de cada abordagem;
- Introduzir as estruturas de dados fundamentais implementadas sobre arranjos (*array's*) com a utilização de exemplos práticos de aplicações.

Orientações de estudo

Para que o conteúdo desta Disciplina seja bem aproveitado e haja maior aplicabilidade na sua formação acadêmica e atuação profissional, siga algumas recomendações básicas:



Assim:

- ✓ Organize seus estudos de maneira que passem a fazer parte da sua rotina. Por exemplo, você poderá determinar um dia e horário fixos como seu “momento do estudo”;
- ✓ Procure se alimentar e se hidratar quando for estudar; lembre-se de que uma alimentação saudável pode proporcionar melhor aproveitamento do estudo;
- ✓ No material de cada Unidade, há leituras indicadas e, entre elas, artigos científicos, livros, vídeos e sites para aprofundar os conhecimentos adquiridos ao longo da Unidade. Além disso, você também encontrará sugestões de conteúdo extra no item **Material Complementar**, que ampliarão sua interpretação e auxiliarão no pleno entendimento dos temas abordados;
- ✓ Após o contato com o conteúdo proposto, participe dos debates mediados em fóruns de discussão, pois irão auxiliar a verificar o quanto você absorveu de conhecimento, além de propiciar o contato com seus colegas e tutores, o que se apresenta como rico espaço de troca de ideias e de aprendizagem.

Introdução

Nesta primeira unidade, vamos ter a oportunidade de estudar as estruturas de dados fundamentais e as respectivas operações que podem ser executadas sobre elas. Os profissionais da computação se dedicam ao desenvolvimento de algoritmos que possam ser utilizados para a resolução de problemas. Os algoritmos são elaborados de modo que possam receber os dados de entrada de um problema e, por meio de uma sequência de instruções, transformar a entrada em uma saída pretendida. Assim, a forma pela qual os dados são estruturados pode facilitar sua manipulação, sendo um aspecto muito importante para o bom funcionamento dos algoritmos.

O computador armazena os dados que serão manipulados pelos programas em uma memória principal. Veja que um programa de computador é a implementação de um algoritmo, em uma linguagem de programação específica. Desse modo, é possível para o programador instruir o computador sobre como ele deve processar os dados que serão recebidos; além disso, o computador deve ser capaz de armazenar os dados, temporariamente, durante todo o processamento.

Os tipos de dados que serão processados definem a quantidade de memória que será necessária para o seu armazenamento. Suponha que se pretenda somar dois números inteiros, assim é necessário que se tenha espaço suficiente para armazenar esse tipo específico de dado. Comumente, para um número inteiro ser armazenado aloca-se um conjunto de quatro *bytes* na memória. Sabemos que um *byte* é um conjunto de oito *bits*, e que um *bit* representa os valores 0 ou 1. Assim, uma sequência de 32 *bits* será reservada para armazenar um número inteiro, o que pode representar uma faixa com 2^{32} valores distintos.

A maioria das linguagens de programação dispõe de um conjunto de tipos de dados, os quais são denominados de tipos de dados primitivos. Além do tipo inteiro (*int*), podemos destacar os números reais (ponto flutuante ou *float*), os caracteres (*char*), os valores lógicos (*boolean*), dentre outras variações possíveis. Cada tipo de dado primitivo demanda uma quantidade específica de *bits* para o seu armazenamento.

As próximas seções farão uma descrição a respeito de estruturas que possam ser utilizadas para armazenar conjuntos de dados primitivos. Ou ainda, estruturas que possam armazenar outras estruturas de dados mais complexas. Inicialmente, vamos realizar uma descrição, superficial, sobre a memória principal e as diferentes formas de alocarmos espaços para armazenamento de dados.

A Memória

O armazenamento de dados em um computador pode considerar diferentes tipos de dispositivos. Dentre os principais dispositivos de armazenamento, podemos destacar as memórias *RAM* (*Random Access Memory*), *ROM* (*Read Only Memory*), *cache*, dentre outros dispositivos possíveis. No contexto de processamento de dados, os dispositivos eletrônicos, em especial o computador, utilizam a *RAM* como memória principal ou

primária. Assim, os dados que serão processados estarão armazenados, de maneira temporária, na memória RAM, visto que se trata de uma memória volátil.

A RAM é um dispositivo de armazenamento eletrônico de dados que suporta as operações de leitura e escrita de dados. Podemos considerar a RAM, daqui em diante referida somente como memória principal, como uma grade bidimensional, na qual cada interseção entre linha e coluna contém uma unidade de armazenamento. A depender do computador considerado, as unidades de memória podem ser compostas por um número variado de *bits*. Na prática, podem haver diversas grades compondo a memória principal; assim, podemos pensar nesse dispositivo como sendo uma estrutura tridimensional de unidades de armazenamento.

A memória principal apresenta algumas características importantes para o seu funcionamento. Trata-se de uma memória de acesso rápido, quando comparado, por exemplo, com o disco rígido (*Hard Disk* ou *HD*) do computador, e cujo acesso é feito por meio de um endereçamento direto de cada unidade de armazenamento. Assim, cada uma das três dimensões da memória (grade, linha e coluna) possui um endereço único que identifica a unidade de armazenamento individualmente.



Você Sabia?

A RAM é uma memória a semicondutor que utiliza um circuito lógico específico, denominado *flip-flop*, para armazenar os *bits*. Nesse sentido, cada unidade de armazenamento é, fundamentalmente, um registrador paralelo.

As unidades de armazenamento podem ser disponibilizadas para uso individualmente ou em conjunto. Quando um programa de computador precisa de um espaço de memória para o armazenamento de dados, ele faz uma alocação de memória. Nesse processo, é feita uma verificação da quantidade de memória demandada e uma busca por um espaço contíguo de memória, que seja suficiente para comportar os dados. Assim, o processo de alocação de memória retornará o endereço físico da primeira unidade de armazenamento alocada.

Existem duas formas de se fazer a alocação de memória, as alocações estática e dinâmica, as quais serão detalhadas em seguida.

Alocação Estática

Um programa de computador, como vimos anteriormente, é a implementação de um algoritmo em uma linguagem de programação. Por sua vez, o programa é submetido a um processo de compilação que resultará em um arquivo com instruções em código de máquina, que pode ser executado no processador. Nesse contexto, um programa em execução é denominado processo; assim, dizemos que as operações realizadas durante o processo são feitas em tempo de execução.

A alocação estática de memória é feita de forma prévia, ou seja, antes da execução propriamente. Esse tipo de alocação ocorre quando, por exemplo, fazemos a declaração de variável global ou estática. Nesse mesmo contexto, quando fazemos a alocação de variáveis

locais ou parâmetros de funções, denominamos de alocação automática. Veja que, no último caso, a alocação de memória é feita quando invocamos uma função específica.

Para ambos os casos descritos, é importante notar que a alocação é feita de maneira prévia, antes do tempo de execução. Desse modo, a alocação estática reserva um espaço de memória contíguo (sequencial) com tamanho previamente definido. Isso implica uma estrutura mais simples de ser gerenciada. Por outro lado, é comum observarmos as unidades de armazenamento ociosas, por conta de um superdimensionamento na alocação, ou a demanda superar a quantidade de memória alocada, devido à dificuldade em se definir, de forma precisa, a quantidade de memória que será necessária.

Alocação Dinâmica

O processo de alocação dinâmica de memória é feito em tempo de execução. Assim, à medida que o processo demanda mais espaço na memória, a alocação de um bloco de memória pré-determinado¹ é feita, de modo a atender especificamente a essa demanda.

A vantagem da alocação dinâmica está em não ser necessário especificar, de forma prévia, a quantidade de memória que será necessária. Desse modo, durante a execução do programa novos blocos de memória podem ser solicitados, à medida que novos dados precisem ser armazenados.

Nesse tipo de alocação, os dados ficam dispersos pela memória, não havendo uma organização explícita entre eles. O encadeamento dos dados é feito por meio de ponteiros. Um ponteiro é uma referência a um endereço físico de uma unidade de armazenamento. Essa forma de localização dos dados na memória pode ser mais complexa de se gerenciar e, em alguns casos, é responsabilidade do programador tanto a alocação quanto a liberação da memória após o uso.

Vetores

Um vetor ou *array* é uma estrutura de dados que armazena, a princípio, dados primitivos desde que sejam do mesmo tipo (estrutura homogênea). Aqui é importante não confundirmos com a classe *Vector* do *Java*, ou com outras linguagens que permitem o uso de um vetor de tipos de dados diferentes. No contexto deste curso, estamos considerando um vetor como sendo uma estrutura homogênea de dados estaticamente alocados. Dessa forma, as vantagens e desvantagens, descritas anteriormente para a alocação estática, são observadas nesse tipo de estrutura. Outro ponto importante de ser observado é que estamos tratando especificamente de vetores unidimensionais, os quais atendem ao nosso propósito inicial. Veja que podemos ter um vetor de vetores, o que caracterizaria um vetor bidimensional (matriz). Nesse sentido, podemos ter vetores de muitas dimensões (multidimensionais), o que, a princípio, não é o nosso foco principal.

A alocação de um vetor é feita a partir da definição do tipo específico de dado que será armazenado, bem como da quantidade de elementos a ser armazenada. Assim, será

¹ Considere um bloco de memória como sendo um conjunto de unidades de armazenamento.

reservada uma quantidade específica de *bits* contíguos na memória principal. O acesso aos elementos no vetor é facilitado por conta de sua organização sequencial na memória.

Considere o armazenamento de um vetor de inteiros positivos $V = [4, 7, 10, 3, 2, 4, 5]$ em memória. Suponha que para cada valor inteiro, será alocado um total de quatro *bytes* e que a unidade de armazenamento, para esse exemplo, é igual a um *byte*. Nesse caso, deverão ser identificados um total de 28 *bytes* que sejam contíguos e estejam livres na memória. O processo de alocação retornará o endereço físico da primeira unidade de armazenamento alocada. Na prática, esse endereço será associado ao nome que foi atribuído ao vetor pelo programador.

A Figura 1 apresenta uma abstração do exemplo anterior. O vetor conta com indexadores (destacados na cor verde na figura) para identificar suas posições. Assim, é possível acessar quaisquer posições no vetor a partir do endereço associado ao seu nome.

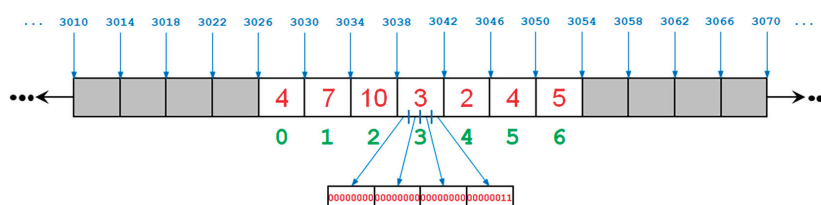


Figura 1 – Abstração de um vetor de inteiros em memória fisicamente endereçada

Fonte: Acervo do Conteudista

Vamos supor que queiramos acessar a posição cujo indexador é cinco. Assim, a partir de uma operação aritmética básica, é possível realizar esse acesso multiplicando o indexador pelo tamanho do tipo de dado e somando esse resultado ao endereço associado ao nome do vetor. Em outras palavras, o indexador que queremos acessar (5) multiplicado pelo tamanho do tipo de dado em *bytes* (4 *bytes* para cada inteiro) resultará na distância, em *bytes*, entre o endereço armazenado e o endereço da posição objetivo ($5 \times 4 = 20$). Somando esse valor ao endereço armazenado, localizamos a posição objetivo ($3026 + 20 = 3046$). O processo descrito é transparente ao programador, ou seja, é realizado automaticamente, sem que haja a necessidade de qualquer intervenção do programador.

As operações sobre um vetor podem ser feitas com a utilização do seu nome e do indexador objetivo. Por exemplo, se desejamos mostrar o conteúdo do vetor na posição quatro, utilizaremos o comando de impressão na saída padrão sobre a posição $V[4]$. Para o caso de estarmos utilizando a linguagem de programação *Java*, a sintaxe ficaria da seguinte forma:

```
System.out.println("%d\n",V[4]);
```

Note que o valor que será apresentado está na posição $V + 4 \times 4$ ou $3026 + 4 \times 4 = 3042$.

Na prática, o acesso às posições de um vetor sempre será feito a partir do seu respectivo nome, associado ao indexador que queremos (o indexador será declarado entre colchetes). O processo, descrito anteriormente, ilustra como as posições são localizadas considerando que a memória é um conjunto de *bits* identificados em blocos; assim, esse processo não é uma preocupação do programador no desenvolvimento de aplicações que utilizem vetores.

Caro(a) aluno(a), este curso tem por objetivo apresentar as estruturas de dados fundamentais e as respectivas operações sobre elas. Veja que não se trata de um curso de programação; assim, as descrições das operações serão feitas em pseudocódigo. Como o próprio nome ilustra, um pseudocódigo não é um código e, portanto, não pode ser introduzido em um computador. A ideia é registrar a lógica da operação, de modo que você possa entender e implementar em uma linguagem de programação de sua preferência. Existe uma enormidade de exemplos codificados, em diferentes linguagens, disponíveis na *internet*. Assim, esperamos que você compreenda os conceitos e, ao final do curso, esteja apto(a) a aplicá-los em diferentes linguagens.

O vetor é uma estrutura de dados estática importante, pois, a partir dele, é possível a implementação de diversas outras estruturas de dados mais sofisticadas. Nesse contexto, vamos seguir com apresentação de duas estruturas simples aplicadas considerando vetores; tratam-se da fila e da pilha, as quais serão tratadas nesta unidade. Entretanto, o vetor será utilizado, novamente, quando formos estudar os algoritmos de busca e de ordenação.



Para se aprofundar, leia este artigo, do professor Paulo Feofiloff (DCC-IME-USP), com exemplos e exercício sobre vetores, considerando a linguagem de programação C para a implementação e manipulação de vetores, disponível em: <https://bit.ly/3rloxuA>

Fila Estática

O vetor, como uma estrutura de dados estática, pode ser utilizado para a implementação de outras estruturas similares. Nesse sentido, a fila é uma dessas estruturas e considera um vetor estático e uma determinada regra de acesso. Uma regra de acesso define a forma pela qual podemos realizar manipulações sobre a estrutura.

Considere uma analogia com uma fila de banco; nesse caso, o cliente que chega primeiro à fila será, também, atendido em primeiro lugar. Essa ideia nos ajuda a definir a regra de acesso a uma fila. Denominamos por *FIFO* (*First In First Out*) a regra que diz: o primeiro que entra é o primeiro que sai. Desse modo, como em uma fila de banco, o primeiro elemento a entrar na fila será o primeiro a sair dela.

Uma fila, considerada sobre um vetor, apresenta dois pontos de acesso. Um ponto de acesso é a cabeça da fila, o qual denominaremos de *head*, e é a partir desse ponto que os elementos são retirados da fila. Outro ponto de acesso é o final da fila, o qual denominaremos de *tail*, onde os elementos entram na fila. Cada um dos pontos de acesso referencia uma posição na fila (ou, nesse caso, um vetor) utilizando os indexadores.

Considere a Figura 2, a qual apresenta uma abstração de uma fila implementada sobre um vetor de inteiros V , estaticamente alocado, de tamanho igual a sete. Veja que os pontos de acesso à fila (*head* e *tail*) são representados por variáveis do tipo inteiro, as quais armazenam as posições correspondentes na fila por meio dos indexadores.

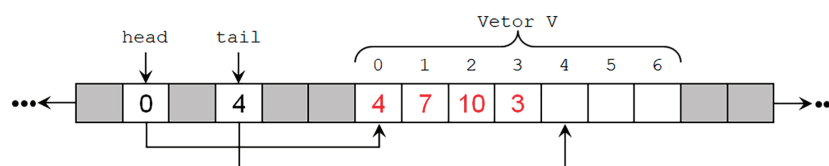


Figura 2 – Abstração de uma fila implementada sobre um vetor de inteiros na memória. As posições *head* e *tail* são o início e o final da fila, respectivamente

Fonte: Acervo do Conteudista

A fila da Figura 2 está parcialmente preenchida com os valores 4, 7, 10 e 3. A variável *head* aponta para o começo da fila, ou seja, ele contém o valor do indexador da posição inicial da fila; nesse caso, o indexador é igual a 4, indicando que o valor 4 está no início da fila. Por outro lado, a variável *tail* contém o valor 4, o qual é o indexador da posição final da fila, é nessa posição que um novo valor será introduzido, caso haja essa demanda.

As estruturas de dados estão associadas a operações possíveis de serem realizadas sobre elas. No caso da fila, podemos ter as operações de inserção e remoção de elementos, as quais serão denominadas de *enqueue* e *dequeue*, respectivamente.

Para realizar a operação *enqueue* (inserção), é necessário que haja espaço na fila para um novo elemento. Ou seja, se a fila estiver cheia não é possível inserir um novo elemento. A fila estará cheia quando a variável *tail* estiver armazenando um valor igual ao tamanho da fila. Veja que, nesse caso, a variável *tail* estará apontando para uma posição fora dos limites da fila.

Algoritmo 1 – Função para verificar se uma fila v está cheia

O Algoritmo 1 descreve os passos para verificar se a fila está cheia. Caso a variável *tail* tenha seu valor igual ao tamanho do vetor ($|V|$), o algoritmo retorna *VERDADEIRO*; caso contrário, retorna *FALSO*. Veja que estamos considerando *tail* como variável global.

$$\text{IsFull}(V)$$

- ```

1. se $tail = |V|$ $\triangleleft |V|$ indica o tamanho de V
2. retorna VERDADEIRO
3. senão
4. retorna FALSO

```

### Algoritmo 2 – Procedimento para inserir o valor $x$ na fila $v$

A operação de inserção é descrita pelo Algoritmo 2. O procedimento *Enqueue()* recebe, como parâmetros, o vetor *V* e um valor *x* que será inserido na fila. Primeiro é feita a verificação para sabermos se a fila está cheia, por meio da função *IsFull()*. Nesse caso, se a função retornar *FALSO* isso indica que a fila não está cheia, utilizamos o operador lógico *!* (*NÃO*) para inverter o valor lógico da função. Caso a fila não esteja cheia, o valor *x* é atribuído à posição *tail* na fila *V*. Veja que, após a inserção, o final da fila deverá ser atualizado, assim é preciso incrementar a variável *tail*, que agora irá indicar a próxima posição.

Diferentemente da função *IsFull()*, o procedimento *Enqueue()* não retorna nenhum valor. Nesse caso, a operação alterará o estado do vetor sobre o qual a fila está sendo implementada. Assim sendo, quando não há retorno de valores, utilizamos a denominação procedimento. Por outro lado, quando há o retorno de algum valor, denominamos função.

Enqueue  $(V, x)$

- ```

1.   se ! IsFull (V)                                ≡ ! indica a operação lógica NÃO
2.       V [tail] ← x
3.       tail ← tail + 1
4.   senão
5.       erro overflow

```

A operação de remoção de um elemento da fila é denominada *dequeue*. Note que, para que seja possível remover um elemento da fila, é preciso que haja ao menos um elemento a ser removido, ou seja, a fila não pode estar vazia. Uma fila está vazia quando as variáveis *head* e *tail* contêm o mesmo valor. Quando inicializamos uma fila, as variáveis *head* e *tail* são inicializadas com o valor 0. Assim, a fila começa e termina na mesma posição, indicando que não há elementos armazenados.

Algoritmo 3 – Função para verificar se uma fila v está vazia

A função *IsEmpty* (), descrita no Algoritmo 3, descreve os passos para que possamos verificar se uma fila está vazia. Nesse caso, não temos a passagem de parâmetros para a função, lembrando que partimos da premissa que as variáveis *head* e *tail* são globais. Veja que caso os valores armazenados nas variáveis *head* e *tail* sejam iguais, a função retorna *VERDADEIRO*; caso contrário, retornará *FALSO*.

IsEmpty()

- ```

1. se head = tail
2. retorna VERDADEIRO
3. senão
4. retorna FALSO

```

### Algoritmo 4 – Função para remover um elemento da fila

A remoção de um elemento na fila é feita na posição indicada pelo ponteiro *head*. A utilização da denominação ponteiro para *head* não é precisa, mas vamos considerar dessa forma, visto que o conteúdo de fato aponta para uma posição no vetor que representa a fila. No contexto das estruturas de dados obtidas por alocação dinâmica de memória, esse conceito é mais preciso, visto que utilizaremos variáveis que armazenam endereços que apontam para diferentes posições de memória.

Dequeue (V)

- ```

1.   se ! IsEmpty ()
2.        $x \leftarrow V[\text{head}]$ 
3.        $\text{head} \leftarrow \text{head} + 1$ 

```

4. **retorna** x
5. **senão**
6. **erro** *underflow*

O Algoritmo 4 descreve os passos que compõem a função *Dequeue* (). Após verificar se a fila não está vazia, o elemento na posição *head* é atribuído a uma variável auxiliar. Assim, incrementa-se o ponteiro *head*, de modo que ele referencie a próxima posição na fila, a qual se tornará seu início. Por fim, a função retorna o elemento, reservado na variável auxiliar, para o ponto no qual a função foi chamada.



Quando tentamos inserir um novo elemento em um fila cheia, temos um erro de *overflow*. Por outro lado, quando tentamos remover um elemento de uma fila vazia, temos um erro de *underflow*.

Existem várias aplicações para a fila, principalmente considerando-se problemas cotidianos, como, por exemplo, controlar o atendimento de pessoas em algum tipo de serviço, gerenciar pedidos que tenham uma mesma prioridade, dentre outros. Uma aplicação no contexto da ciência de computação é a utilização de uma fila para o cálculo de distância em grafos.

Há, ainda, no contexto de implementação, as operações de criação e de inicialização de uma fila. Essas operações referem-se à alocação e à inicialização de memória, considerando a estrutura que se pretende utilizar. Veja que não estamos objetivando a implementação em linguagem de programação das estruturas de dados apresentadas, mas sim o entendimento a respeito de seu funcionamento, por meio do estudo de seus conceitos básicos.

Um olhar mais atento para a fila, conforme foi descrita até aqui, e sobre o seu funcionamento, notará que há um problema nessa implementação. Suponha uma fila V , cujo estado é descrito na Figura 3. Veja que, nessa representação, a fila está cheia se considerarmos o conceito definido anteriormente para esta condição, sendo o valor atribuído à variável *tail* igual ao tamanho do vetor que implementa a fila – no caso, sete.

A operação de inserção de um novo elemento na fila, representada na Figura 3, irá gerar um erro de *overflow*; no entanto, veja que há posições no vetor que estão disponíveis, por conta da remoção de elementos, no início da fila. Assim, verifica-se que, apesar de haver espaço para o armazenamento de novos elementos, a fila não poderá receber novos valores.

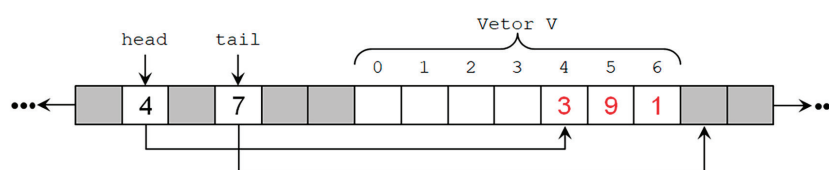


Figura 3 – Abstração de uma fila implementada sobre um vetor de inteiros na memória. O estado atual é de fila cheia, pois $tail = |V|$

Fonte: Acervo do Conteudista

O problema descrito poderia ser resolvido se, a cada remoção de elementos na fila, fizéssemos com que os elementos posteriores fossem realocados uma posição no sentido do início da fila. Desse modo, a cada remoção uma nova posição seria liberada no final da fila, resolvendo, assim, o problema das posições ociosas.

A solução descrita para o problema, apesar de factível, é pouco viável. Toda operação de remoção de elementos da lista seria seguida de uma série de outras operações para o deslocamento de todos os elementos. Caso a fila tivesse um número muito grande de elementos, o gasto de tempo seria impactante. Na próxima seção, veremos uma solução alternativa para o problema das posições ociosas, a qual é denominada de fila circular.

Fila Estática Circular

Vamos iniciar esta seção com uma revisão sobre operações aritméticas, especificamente a divisão inteira. Quando consideramos a divisão entre dois números inteiros e queremos representar o resultado dessa operação, também, com um número inteiro, é comum termos um resto diferente de zero. Assim, sejam a (dividendo) e b (divisor) números inteiros positivos, a divisão inteira ($a \div b$) resultará em um número, também inteiro, c (quociente) e em um resto da divisão, aqui representado por d . Nesse contexto, as operações $a \div b = c$ e $a \% b = d$ (leia-se a módulo b) são divisões as quais resultam no quociente e no resto da divisão, respectivamente.

A fila circular resolve o problema das posições ociosas, sem que seja necessário realizar movimentações adicionais na fila. Assim, o limite de armazenamento será igual ao tamanho do vetor considerado para a implementação da fila.

As operações, em uma fila circular, seguem de forma similar ao descrito para a fila estática. Porém, há a necessidade de algumas adaptações. Primeiro, para evitar os erros de *underflow* e *overflow*, as operações *IsFull()* e *IsEmpty()* utilizarão um contador (*count*) para as verificações de preenchimento do vetor. Assim, a fila estará cheia quando $count = |V|$, ou seja, quando o contador for igual ao tamanho do vetor. Veja que o contador deverá ser incrementado sempre que um novo elemento for inserido na fila. Por outro lado, sempre que um elemento for removido da fila, o contador deverá ser decrementado. Desse modo, a fila estará vazia quando $count = 0$; em outras palavras, o valor zero armazenado no contador indica que não há elementos na fila.

O procedimento *Enqueue()* fará a inserção de um novo elemento da fila na posição $V[tail \% |V|]$. Veja que consideramos a variável *tail* da mesma maneira que no procedimento anterior, porém o valor do indexador no vetor será dado pelo resto da divisão inteira entre *tail* e o tamanho do vetor. Além disso, após a inserção, as variáveis *tail* e *count* devem ser incrementadas em uma unidade, de modo que o final da fila seja atualizado e que contabilizemos mais um elemento inserido.

Finalmente, a função *dequeue()* será modificada de modo a retornar o elemento armazenado na posição $V[head \% |V|]$, ou seja, o elemento retornado será aquele que ocupa a posição indicada pelo resto da divisão inteira entre a variável *head* e o tamanho do vetor. De forma similar à descrita anteriormente, a variável *head* deverá

ser incrementada em uma unidade, indicando, assim, que o início da fila foi alterado. Porém, como temos menos um elemento na fila, a variável *count* deverá ser decrementada em uma unidade, ajustando, assim, a quantidade de elementos na fila.

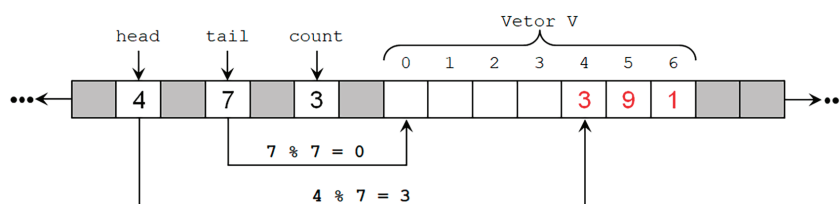


Figura 4 – Abstração de uma fila circular implementada sobre um vetor de inteiros na memória. O estado atual permite a inserção de elementos enquanto $\text{count} < |V|$

Fonte: Acervo do conteudista

Considere o estado da fila, representada na Figura 3, a qual não implementa a lógica descrita para uma fila circular, em comparação com a fila circular, representada na Figura 4. Veja que, para o último estado representado na Figura 4, há a possibilidade de inserção de novos elementos, utilizando as posições iniciais do vetor que estão ociosas. Note que, quando utilizamos a operação módulo para indicar as posições no vetor, o final da fila (*tail*) passa a ser a posição 0, em vez de 7 que é o conteúdo, de fato, da variável.

Uma observação importante aqui é que podemos otimizar o armazenamento na fila até que, realmente, ela esteja completamente preenchida, sem a necessidade de movimentações adicionais de elementos. Conforme descrito anteriormente, as movimentações fazem com que o processo seja mais lento que o necessário.



Modifique os algoritmos 1, 2, 3 e 4 de modo que eles reflitam a ideia de uma fila circular. Além disso, escolha uma linguagem de programação de sua preferência e implemente uma fila circular.

Na próxima seção estudaremos outra estrutura de dados, implementada sobre um vetor, que apresenta outra regra de acesso, na qual utilizaremos um único ponto para realizar as operações de inserção e remoção de elementos. Trata-se da pilha, que é uma das estruturas fundamentais, a qual pode ser considerada em diferentes aplicações.

Pilha Estática

A pilha é uma estrutura de dados na qual as operações são realizadas em um único ponto. As operações básicas que podem ser realizadas sobre uma pilha são a inserção e a remoção de elementos, denominaremos essas operações de *push* e *pop*, respectivamente.

A regra de acesso a uma pilha descreve que os elementos serão retirados na ordem inversa em que foram inseridos. Essa regra é denominada *LIFO* (*Last In First Out*) e significa que o último que entra será o primeiro a sair. Por analogia, podemos considerar uma pilha de pratos, veja que não conseguimos retirar o primeiro prato que foi inserido

(que está na base da pilha), temos acesso somente ao último prato inserido; assim, para retirar o primeiro, antes teremos que retirar todos os demais.

O ponto de acesso à pilha é denominado *topo*, onde todas as operações são realizadas. Assim, os elementos serão inseridos no topo da pilha e a remoção só será possível sobre o elemento que, também, está posicionado no topo.

A Figura 5 apresenta uma abstração de uma pilha em diferentes estados. Veja que o armazenamento de uma pilha em memória segue a mesma representação utilizada para a fila. Porém, para contribuir com o entendimento sobre a lógica funcional de uma pilha, utilizou-se a representação do vetor, sobre o qual a pilha é implementada, na vertical, ficando, assim, o topo na parte superior e a base na parte inferior da representação.

O estado (a), representado na Figura 5, descreve uma pilha vazia, que é resultado da operação de inicialização. Veja que essa operação é uma questão de implementação, onde se realiza a alocação de um vetor, para armazenar os elementos da pilha, e uma variável que armazenará a posição referente ao topo da pilha. Uma pilha estará vazia quando a variável topo armazenar o valor zero.

Os estados de (b) até (g) são resultado da operação *push*, que insere um valor no topo da pilha. O topo é indicado por meio de uma variável que armazena a respectiva posição do vetor. Após a inserção de seis elementos na pilha, o estado (g) descreve uma pilha cheia. Assim, podemos observar que a indicação de uma pilha cheia é quando a variável topo armazena um valor igual ao tamanho do vetor.

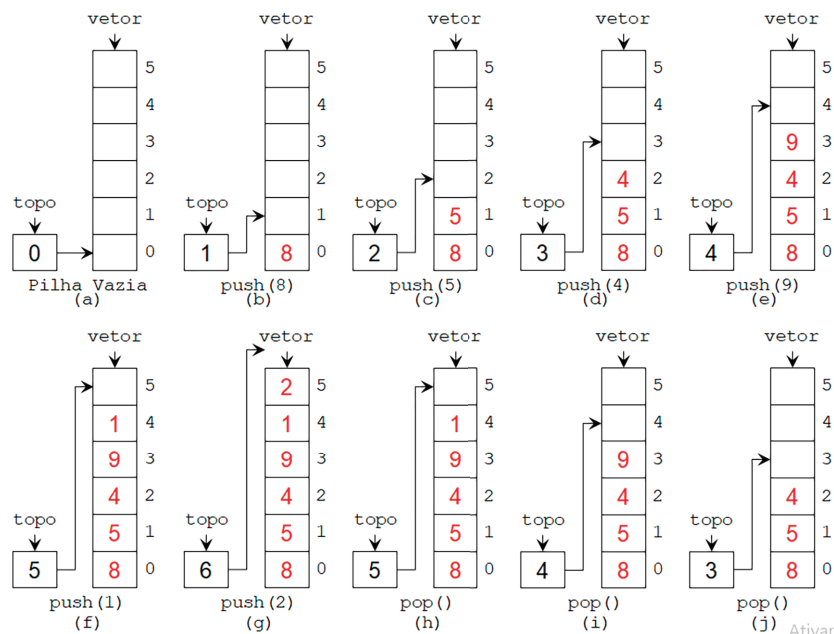


Figura 5 – Abstração de uma pilha implementada sobre um vetor de inteiros.

Cada um dos estados corresponde ao resultado das operações *push* e *pop*

Fonte: Acervo do Conteudista

Os estados de (h) até (j) são resultado da operação *pop*, essa operação retorna o valor armazenado na posição indicada pela variável topo e decrementa essa variável em uma unidade, atualizando, assim, o topo da pilha.

Algoritmo 5 – Função para verificar se uma pilha está cheia

Para evitar erros de *underflow* e *overflow*, deve-se checar o estado da pilha antes da realização das operações *push* ou *pop*. A operação *push*, responsável pela inserção de elementos, deve ser realizada somente se houver espaço para o armazenamento. Assim, deve-se verificar se a pilha está ou não cheia. Vimos que uma fila está cheia quando $topo = |V|$; assim, o Algoritmo 5 descreve os passos para a implementação da função *IsFull ()* no contexto de uma pilha, implementada sobre um vetor de inteiros.

IsFull (V)

1. **se** $topo = |V|$
2. **retorna** VERDADEIRO
3. **senão**
4. **retorna** FALSO

Algoritmo 6 – Procedimento para inserir o valor x na pilha

A função *IsFull ()*, descrita no Algoritmo 5, verifica se o valor armazenado na variável *topo* é igual ao tamanho do vetor que armazena os elementos da pilha e retorna *VERDADEIRO*, caso a operação lógica se confirme, ou *FALSO*, caso contrário. Conforme descrito anteriormente, essa verificação é importante para evitar erro de *overflow*.

Após a verificação da ocupação da pilha, a operação *Push ()* pode ser executada para a inserção de elementos nessa estrutura. Essa operação recebe um elemento a ser inserido na pilha, na posição estabelecida pelo valor da variável *topo*, atualizando, em seguida, essa variável. O Algoritmo 6 descreve os passos que compõem essa operação, lembrando que estamos considerando a variável *topo* como global.

Push (x, V)

1. **se** $\neg IsFull (V)$
2. $V [topo] \leftarrow x$
3. $topo \leftarrow topo + 1$
4. **senão**
5. **erro** *overflow*

Algoritmo 7 – Função para verificar se uma pilha está vazia

Conforme descrito anteriormente, não podemos realizar a inserção de um elemento na pilha se ela estiver cheia. Da mesma forma, a retirada de um elemento da pilha está condicionada à existência de, pelo menos, um elemento que possa ser retirado. Nesse contexto, é preciso verificar se a pilha está vazia antes de proceder com a retirada de elementos. A função que é responsável por essa verificação é descrita no Algoritmo 7.

Vale lembrar que a pilha estará vazia quando a variável *topo* armazenar o valor 0. Assim, caso essa condição seja satisfeita, a função retornará *VERDADEIRO*; caso contrário, será retornado o valor lógico *FALSO*.

IsEmpty ()

1. **se** topo = 0
2. retorna VERDADEIRO
3. **senão**
4. retorna FALSO

Algoritmo 8 – Função para remover o valor posicionado no topo da pilha *v*

A operação de remoção, aqui denominada *pop*, retira da pilha o elemento que está armazenado na posição indicada pela variável *topo*, após a verificação da existência de elementos na pilha, a qual é feita pela função *IsEmpty*. Assim, o Algoritmo 8 descreve os passos que compõem a função *Pop*.

Pop (*V*)

1. **se** ! *IsEmpty* ()
2. $x \leftarrow V[\text{topo}]$
3. topo $\leftarrow$ topo - 1
4. **retorna** *x*
5. **senão**
6. **erro** *underflow*

Existem diversas aplicações para as pilhas, principalmente no contexto da computação. A recursão é um exemplo de aplicação não explícito, estudaremos esse conceito nas próximas unidades. Quando utilizamos um algoritmo recursivo, ele vai empilhar as chamadas às instâncias maiores até que se possa chegar a uma instância pequena, a qual seja resolvida diretamente. Assim, as chamadas empilhadas serão retiradas da pilha na ordem inversa à ordem em que entraram.

Outro exemplo é decidir se uma expressão numérica, estruturada com parênteses, colchetes e chaves, está bem formulada. Veja que, para cada um dos símbolos esquerdos (, [, e {, devemos ter o seu correspondente direito, fechando o par. Assim, o exemplo abaixo está bem formulado:

{[([])[()]]},

Enquanto o exemplo seguinte não está bem formulado:

{({}[[]]}).

O algoritmo que decide sobre esse problema é simples e utiliza uma pilha em sua solução.

Considere que os símbolos esquerdos lidos serão empilhados e quando um símbolo direito é lido, o algoritmo verifica se ele é simétrico ao elemento no topo da pilha. Ou seja, se no topo está o símbolo [e o próximo símbolo lido é], então, como eles são simétricos, o algoritmo retira o símbolo do topo e realiza a leitura de um novo símbolo.

Se ao final da entrada a pilha estiver vazia, o algoritmo aceita a entrada, decidindo que a sequência de símbolos é bem formulada, ou a rejeita, caso contrário.

Vimos nessa unidade introdutória que as estruturas de dados, particularmente as lineares, têm papel importante na construção dos algoritmos e na forma como os dados serão armazenados em memória. Além disso, conhecemos as estruturas de dados fundamentais, a fila e a pilha. Essas estruturas serão importantes para diversas aplicações futuras que possam ser necessárias.

Na próxima unidade, veremos os principais algoritmos de ordenação ou classificação e, também, sobre a busca de elementos específicos em conjuntos ordenados ou não.

Material Complementar

Indicações para saber mais sobre os assuntos abordados nesta Unidade:

Vídeos

O que são Estruturas de Dados?

https://youtu.be/Frkc_otGrGU

Estrutura de Dados – Fila Estática – exemplo de implementação

https://youtu.be/Qs_ryxhyx_w

Pilha Estática (Vetores) FÁCIL E COMPLETO! | Estrutura de Dados | Programação em C | Pixel Tutoriais

<https://youtu.be/6DeriNEszk0>

Leitura

Uma fila com duas pilhas

<https://bit.ly/378zHRv>

Referências

CORMEN, T. H. *et al.* **Introduction to algorithms**. MIT press, 2009.

GOODRICH, M. T. **Estruturas de dados & algoritmos em Java**. 5ª ed. Porto Alegre: Bookman, 2013. (*e-book*)

SZWARCFITER, J. L. **Estruturas de dados e seus algoritmos**. 3ª ed. Rio de Janeiro: LTC, 2010. (*e-book*)



Cruzeiro do Sul
Educatonal